# FUNDAMENTAL R PROGRAMMING

*Syntax, Data Structures, Control Flow & Scoping Scenarios*

Welcome to the official **Marlin Tutors** R Programming Guide. This comprehensive handbook is designed specifically for students and professionals looking to establish a rigorous foundation in R. From basic syntax and operations to advanced functions, recursion, and global scope scenarios, this manual walks you through actual executable codes, detailed subject matter explanations, and structured classroom exercises to build mastery.

---

***Instructor Copy***
*Published: September 2026*
*Course Level: Absolute Beginner to Intermediate*

# TABLE OF CONTENTS

# 1. Language Overview & Intro (R HOME & R Intro)

R is a programming language and software environment specifically developed for **statistical computing, data analysis, and graphical presentation**. It is widely used by statisticians, data scientists, researchers, and bioinformaticians across various industries like finance, bioinformatics, and academic research.

### Origins & Open-Source Legacy

R was first created by **Robert Gentleman and Ross Ihaka in 1991** at the University of Auckland in New Zealand. The name 'R' represents both a play on its predecessor (the S language) and is derived from the first letter of both authors' names. The language was officially released publicly in 1993, and since the year 2000, it has been maintained under the GNU General Public License as an open-source, highly extensible ecosystem with over 22,000 packages stored in the CRAN (Comprehensive R Archive Network) repository.

# 2. Installation & Setup (R Get Started)

R is an **interpreted language**, which means you write commands, and the interpreter executes them immediately. This makes R exceptionally friendly for experimental computing and quick testing.

### Getting R and RStudio Up and Running

- **Step 1: Download R**: Go to the official CRAN website (https://cran.r-project.org/) to download and install R for your specific operating system (Windows, macOS, or Linux).
- **Step 2: Download RStudio (IDE)**: RStudio is a free, powerful, and highly recommended integrated development environment (IDE) for R, which you can download from Posit (https://posit.co/downloads/).
- **Step 3: Run your first command**: Open the R Console or RStudio and write a simple print or calculation expression. Try typing the following directly into your console:

```
print("Hello World!")
5 + 5
```

**EXPLANATION**

*In R, you can print strings to the screen with the print() function, or simply by typing the string literal directly into the console. Calculations are performed and output immediately. The bracketed prefix '[1]' indicates that the output is the first element of a vector.*

# 3. Core R Syntax & Comments

Writing clean, readable code is essential in R. Statements in R are generally written without semicolons at the end of lines, although semicolons can be optionally used to separate multiple statements on a single line.

## Key Syntax Rules

- **Case Sensitivity**: R is completely case-sensitive. The variable my_var is distinct from my_Var.
- **Primary Assignment**: The arrow operator <- is the primary and preferred variable assignment operator in R (e.g., x <- 10), though the equals symbol = is also supported.
- **Script Files**: Scripts containing series of R commands are saved with a .R extension and run on the console using: Rscript FileName.R.

## Writing Comments in R

Comments are used to write explanatory notes for human readers or to temporarily disable specific lines of code. They are completely ignored by the R interpreter. Comments start with a hash symbol (#) and can span across multiple lines by starting each line with a #.

```
# This is a single-line comment
print("Hello from Marlin Tutors!") # This is an inline comment

# R does not support native multi-line comments.
# To write multiple lines, you must prepend
# each line with a hash symbol (#).
```

# 4. Variables & Concatenation

Variables are named containers used to store data. In R, variables do not need to be declared with any specific type, and they can dynamically change their type after they have been initialized.

### Variable Naming Rules

A valid variable name in R must adhere to the following rules:

- Must start with a letter, or a dot (.).
- If it starts with a dot, it **cannot** be followed immediately by a number.
- Can contain letters, numbers, dots (.), and underscores (_).
- Cannot start with a number or an underscore, nor contain special characters like spaces, dashes, or symbols.

### Multiple Variable Assignment & Concatenation

To combine strings or variables, R uses the paste() or paste0() function. The paste() function concatenates arguments with a default space separator, while paste0() concatenates them with no intervening spaces.

```
# Assigning the same value to multiple variables on a single line
var1 <- var2 <- "Apple"

# Concatenation examples
first_name <- "Marlin"
last_name <- "Tutors"

full_name <- paste(first_name, last_name) # Results in: "Marlin Tutors"
no_spaces <- paste0(first_name, last_name) # Results in: "MarlinTutors"
```

# 5. R Data Types & Coercion

R is dynamically typed, but every object has an underlying class. Understanding data types is critical for avoiding calculations errors.

| Data Type | Description | Example Code | Underlying Class |
|---|---|---|---|
| numeric | Decimal numbers (the default type for any number) | x <- 10.5 | numeric |
| integer | Whole numbers; must end with an 'L' suffix | y <- 5L | integer |
| character | Textual strings; wrapped in single or double quotes | z <- 'R Tutorial' | character |

| Data Type | Description | Example Code | Underlying Class |
|---|---|---|---|
| logical | Boolean logical values; either TRUE (T) or FALSE (F) | is_active <- TRUE | logical |
| complex | Complex numbers with real and imaginary parts | comp <- 3 + 5i | complex |
| raw | Stores raw bytes of data | raw_val <- charToRaw('R') | raw |

## Checking Class and Explicit Conversion (Coercion)

You can inspect an object's class using the class() function. To convert from one data type to another, use the corresponding coercion functions: as.numeric(), as.integer(), as.character(), and as.logical().

```
x <- 12L
print(class(x)) # [1] "integer"

# Explicit Conversion
a <- as.numeric(x)   # Convert integer 12L to numeric 12
b <- as.character(x) # Convert integer 12L to character "12"
c <- as.logical(0)   # Convert numeric 0 to logical FALSE
print(class(b))      # [1] "character"
```

# 6. Numbers, Math, and Booleans

R provides comprehensive built-in support for numerical computation and boolean logic evaluation.

### Essential Built-In Math Functions

Apart from standard operators, R provides powerful mathematical utilities:

| Function | Description | Example Input | Example Output |
|---|---|---|---|
| max(...) | Finds the highest value in a set or vector | max(5, 10, 15) | 15 |
| min(...) | Finds the lowest value in a set or vector | min(5, 10, 15) | 5 |
| sqrt(x) | Calculates the square root of a non-negative number | sqrt(16) | 4 |
| abs(x) | Calculates the absolute (positive) value of a number | abs(-7.5) | 7.5 |
| ceiling(x) | Rounds a decimal number UP to the nearest integer | ceiling(4.2) | 5 |
| floor(x) | Rounds a decimal number DOWN to the nearest integer | floor(4.8) | 4 |

### Booleans and Logical Evaluations

Booleans in R represent conditional truth and are represented by TRUE and FALSE. When comparing two values, the expression is evaluated and a logical answer is returned.

```
10 > 9  # Returns TRUE
10 == 9 # Returns FALSE
10 < 9  # Returns FALSE
```

# 7. R Strings & Escape Sequences

A string is used to represent text data. In R, strings must be enclosed in either single or double quotes. To manipulate strings and print special characters, R provides string utilities and backslash escapes.

### String Utility Functions

Two essential string functions to remember are:

- nchar(string): Returns the number of characters in a string (unlike length() which returns vector size).

- grepl(pattern, string): Searches for a substring or pattern, returning TRUE if found and FALSE otherwise.

```
text <- "Marlin Tutors R Tutorial"
print(nchar(text))       # [1] 25
print(grepl("Tutors", text)) # [1] TRUE
```

## Common Escape Characters

The backslash character (\) is used to escape special syntax inside strings. Here are the common escape sequences:

| Escape Sequence | Meaning / Result | Code Example | Output |
|---|---|---|---|
| \n | Inserts a newline character | cat("Hello\nWorld") | Hello World |
| \t | Inserts a horizontal tab | cat("A\tB") | A B |
| \\ | Inserts a literal backslash | cat("C:\\Users") | C:\Users |
| \" | Inserts a literal double quote | cat("He said \"Hi\"") | He said "Hi" |
| \' | Inserts a literal single quote | cat('It\'s R') | It's R |

# 8. Operators (Arithmetic, Relational, Logical, Assignment)

Operators perform mathematical, logical, comparison, and assignment operations in your program.

## Categories of Operators

**1. Arithmetic Operators** (Act element-wise on vectors or single numbers):

| Operator | Description | Example (a <- 10, b <- 3) | Result |
|---|---|---|---|
| + | Addition | a + b | 13 |
| - | Subtraction | a - b | 7 |
| * | Multiplication | a * b | 30 |
| / | Division | a / b | 3.333333 |
| ^ or ** | Exponentiation (raise to power) | a ^ b | 1000 |
| %% | Modulus (Remainder of division) | a %% b | 1 |
| %/% | Integer division (discards remainder) | a %/% b | 3 |

**2. Relational / Comparison Operators** (Yield logical values):

| Operator | Description | Example (x <- 5, y <- 8) | Result |
|---|---|---|---|
| < | Less than | x < y | TRUE |
| > | Greater than | x > y | FALSE |
| <= | Less than or equal to | x <= 5 | TRUE |
| >= | Greater than or equal to | y >= 10 | FALSE |
| == | Equal to | x == y | FALSE |
| != | Not equal to | x != y | TRUE |

**3. Logical Operators** (Evaluate multiple conditions):

| Operator | Description | Behavior |
|---|---|---|
| & | Element-wise Logical AND | Compares all elements of vectors element-by-element. |
| \| | Element-wise Logical OR | Compares all elements of vectors element-by-element. |
| && | Single-logical short-circuiting AND | Evaluates only the first element. Best for standard single-condition if blocks. |

| Operator | Description | Behavior |
|---|---|---|
| \|\| | Single-logical short-circuiting OR | Evaluates only the first element. Best for standard single-condition if blocks. |
| ! | Logical NOT | Negates the logical value (converts TRUE to FALSE, and vice-versa). |

**4. Assignment Operators**:

| Operator | Description | Example |
|---|---|---|
| <- | Left assignment (Standard preferred in R) | x <- 10 |
| -> | Right assignment | 10 -> x |
| = | Basic assignment (Standard in other languages) | x = 10 |
| <<- | Global assignment (Assigns/overwrites variable in global environment) | x <<- 10 |

# 9. Control Flow: If...Else Conditionals

Conditional statements let you execute specific blocks of R code depending on whether certain conditions evaluate to TRUE or FALSE.

### Basic and Nested If...Else Structure

R supports standard if, else if, and else keywords. You can combine multiple criteria inside a single conditional block using short-circuiting logical operators (&& and ||).

```
score <- 85

if (score >= 90) {
    print("Grade: A")
} else if (score >= 80) {
    print("Grade: B")
} else {
    print("Grade: C")
}

# Using logical operators inside condition
age <- 20
has_permit <- TRUE

if (age >= 18 && has_permit) {
    print("Eligible to drive!")
} else {
    print("Not eligible to drive.")
}
```

# 10. Control Flow: While & For Loops

Loops allow you to run a block of code repeatedly. R supports two primary loop structures: while loops and for loops.

### The While Loop & Loop Control Statements

A while loop continues executing as long as its condition remains TRUE. Inside any loop, you can use two special control statements:

- **break**: Immediately terminates the loop execution.
- **next**: Skips the remaining code in the current iteration and jumps immediately to the next cycle.

```
i <- 1
while (i <= 6) {
    if (i == 3) {
        i <- i + 1
        next # Skips printing 3 and moves to next iteration
    }
    if (i == 5) {
        break # Stops loop completely when i reaches 5
    }
    print(i)
    i <- i + 1
}
```

## The For Loop & Nested Loops

A for loop iterates over a sequence (such as a vector, list, or matrix). You can nest loops inside other loops to iterate across multi-dimensional grids.

```
# Simple For Loop iterating over a vector of fruits
fruits <- c("Apple", "Banana", "Cherry")
for (fruit in fruits) {
    print(fruit)
}

# Nested Loop Example: Coordinate Grid
for (row in 1:2) {
    for (col in 1:3) {
        cat("Row:", row, "Col:", col, "\n")
    }
}
```

# 11. R Functions & Nested Environments

A function is a reusable block of code designed to perform a specific task. In R, functions are treated as first-class objects and are defined using the function() keyword.

### Declaring and Invoking Functions

Functions can accept arguments, specify default values, and return output values using the return() function. If no explicit return is specified, R automatically returns the value of the last evaluated expression inside the function body.

```
multiply_nums <- function(x = 5, y = 2) {
    result <- x * y
    return(result)
}

# Call with customized arguments
print(multiply_nums(10, 3)) # Output: 30

# Call utilizing default arguments
print(multiply_nums())     # Output: 10
```

### Nested Functions and Lexical Scoping

In R, you can define a function inside another function. These are called **nested functions**. Because R uses **lexical scoping**, a nested function can freely read and access variables defined in its parent function's environment (the enclosing environment).

```
outer_func <- function(x) {
    parent_var <- 10

    inner_func <- function(y) {
        # Can access parent_var and argument x from outer_func!
        ans <- x + y + parent_var
        return(ans)
    }

    return(inner_func)
}

# Run outer_func, which returns inner_func as a callable object
my_nested_call <- outer_func(5)
print(my_nested_call(3)) # Evaluates: 5 + 3 + 10 = 18
```

# 12. Recursion in R (Deep-Dive)

Recursion is an advanced programming technique where a function calls itself directly or indirectly to solve smaller instances of a problem. It exploits R's function stack frames.

## The Architecture of a Recursive Function

To prevent infinite loops that would consume all active memory, every recursive function must have a defined **base case** (a stopping condition). When the base case is satisfied, the function returns a trivial value and begins unwinding the function stack.

## Classic Recursion Example 1: Factorial

The factorial of a non-negative integer n (written as n!) is the product of all positive integers less than or equal to n. The base case is when n is 0 or 1, which returns 1. Otherwise, n is multiplied by the factorial of n - 1.

```
rec_fac <- function(n) {
    if (n == 0 || n == 1) {
        return(1) # Base Case
    } else {
        return(n * rec_fac(n - 1)) # Recursive Step
    }
}

print(rec_fac(5)) # Output: 120
```

## Recursion Example 2: Sum of Series of Numbers

Recursion is highly useful for mathematical sequences, such as computing the sum of the first n integers:

```
sum_n <- function(n) {
    if (n == 1) {
        return(1) # Base Case
    } else {
        return(n + sum_n(n - 1)) # Recursive Step
    }
}

print(sum_n(5)) # Output: 15 (which is 5 + 4 + 3 + 2 + 1)
```

# 13. Global and Local Scoping Scenarios

Understanding variable scope is crucial in R. Variables created outside a function live in the **Global Environment**. Variables created inside a function are **Local Variables** and only exist inside that function's temporary environment.

### Reading Global Variables vs. Local Definition

A function can read a global variable if no local variable with the same name exists. However, if you write a normal assignment (<-) inside a function, it will create a new **local** variable, leaving the global variable untouched.

```
global_var <- 100

test_func <- function() {
    # Read global_var
    print(global_var) # Prints 100

    # Try to modify global_var locally
    global_var <- 50
    print(global_var) # Prints 50 (This is a local variable!)
}

test_func()
print(global_var) # Prints 100 (The global variable remains unchanged!)
```

### Writing to Global Variables with the <<- Operator

To modify or define a global variable from inside a local function environment, you must use the **global assignment operator** (<<-). R will search through parent environments up to the global environment to find and update that variable. If it cannot find the variable, it will create a new one in the global environment.

```
score <- 10

increase_score <- function() {
    # Modify the global variable "score"
    score <<- score + 5
}

increase_score()
print(score) # Prints 15! (Global variable was modified successfully)
```

# 14. Marlin Tutors Practice Exercises

Test your students' comprehension with the following hands-on programming problems. Instruct them to write these codes inside R script files (with .R extensions) and execute them.

### Exercise 1: Basic Math & Concatenation

Write an R script that does the following:
1. Assign the numeric value 16 to a variable called num_val.
2. Calculate its square root, round up any decimal, and save it to a variable called final_result.
3. Use paste() to concatenate the text 'The result is' with final_result and print it.

### Exercise 2: Checking Types & Coercion

Create a variable containing the string '55'. Inspect its class. Then, explicitly convert this string into an integer. Verify that the new variable's class is 'integer' and print its value multiplied by 2.

### Exercise 3: Loop and Skip Control Flow

Write a while loop that starts an iterator x at 1 and counts up to 10. Inside the loop, if x is an even number, use the next statement to skip printing it. Only odd numbers (1, 3, 5, 7, 9) should be printed on the screen.

### Exercise 4: The Scope and Global Override

Initialize a global variable teacher_brand <- 'Marlin Tutors'. Write a function named update_brand() that modifies this global variable to 'Marlin Tutors Premium' using the correct global assignment operator. Call the function and print the global variable to verify it has changed.

### Exercise 5: Mathematical Exponent Recursion

Write a recursive function named recursive_power(base, exponent). It should calculate base raised to the power of exponent recursively (e.g. recursive_power(4, 3) should evaluate as 4 * recursive_power(4, 2)).
*Hint: The base case is when exponent is 0, which returns 1.*

# 15. Practice Exercise Detailed Solutions

These reference codes represent the correct, idiomatic R implementations for the preceding exercises. Use them to grade student submissions or for self-guided study.

### Solution 1: Basic Math & Concatenation

```
num_val <- 16
root_val <- sqrt(num_val)
final_result <- ceiling(root_val)
output_string <- paste("The result is", final_result)
print(output_string)
# Expected Output:
# [1] "The result is 4"
```

### Solution 2: Checking Types & Coercion

```
char_val <- "55"
print(class(char_val)) # [1] "character"

# Coerce to integer
int_val <- as.integer(char_val)
print(class(int_val))   # [1] "integer"

# Multiply and print
result <- int_val * 2
print(result)          # [1] 110
```

### Solution 3: Loop and Skip Control Flow

```
x <- 1
while (x <= 10) {
    if (x %% 2 == 0) {
        x <- x + 1
        next # Skip even numbers
    }
    print(x)
    x <- x + 1
}
# Expected Output:
# [1] 1
# [1] 3
# [1] 5
# [1] 7
# [1] 9
```

### Solution 4: The Scope and Global Override

```
teacher_brand <- "Marlin Tutors"

update_brand <- function() {
    # Use <<- for global assignment
    teacher_brand <<- "Marlin Tutors Premium"
}

update_brand()
print(teacher_brand)
# Expected Output:
# [1] "Marlin Tutors Premium"
```

### Solution 5: Mathematical Exponent Recursion

```
recursive_power <- function(base, exponent) {
    if (exponent == 0) {
        return(1) # Base case
    } else {
        return(base * recursive_power(base, exponent - 1)) # Recursive step
    }
}

# Test the recursive power function
print(recursive_power(4, 3))  # 4^3 = 64
print(recursive_power(10, 4)) # 10^4 = 10000
# Expected Output:
# [1] 64
# [1] 10000
```